

```

# =====
# ADA BOOSTING IMPLEMENTATION
# =====

# Step 1: Import Libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
from mlxtend.plotting import plot_decision_regions

# -----
# Step 2: Create Dataset
# -----

df = pd.DataFrame()

df['X1'] = [1,2,3,4,5,6,6,7,9,9]
df['X2'] = [5,3,6,8,1,9,5,8,9,2]
df['label'] = [1,1,0,1,0,1,0,1,0,0]

print(df)

# -----
# Step 3: Visualize Dataset
# -----

sns.scatterplot(x=df['X1'], y=df['X2'], hue=df['label'])
plt.title("Dataset Visualization")
plt.show()

# -----
# Step 4: Initialize Weights
# -----

df['weights'] = 1 / df.shape[0]

# Features and Labels
X = df.iloc[:,0:2].values
y = df.iloc[:,2].values

# -----
# Step 5: Train First Weak Learner
# -----

dt1 = DecisionTreeClassifier(max_depth=1)

dt1.fit(X, y)

# Prediction
df['y_pred'] = dt1.predict(X)

print(df)

# -----
# Step 6: Plot Decision Tree
# -----

plt.figure(figsize=(6,4))
plot_tree(dt1, filled=True)
plt.show()

# -----
# Step 7: Plot Decision Region
# -----

plot_decision_regions(X, y, clf=dt1, legend=2)
plt.show()

# -----
# Step 8: Calculate Model Weight
# -----

def calculate_model_weight(error):
    return 0.5 * np.log((1-error) / error)

# Assume error = 0.3
alpha1 = calculate_model_weight(0.3)

```

```

def update_row_weights(row, alpha=0.423):
    if row['label'] == row['y_pred']:
        return row['weights'] * np.exp(-alpha)
    else:
        return row['weights'] * np.exp(alpha)

df['updated_weights'] = df.apply(update_row_weights, axis=1)

# Normalize Weights
df['normalized_weights'] = (
    df['updated_weights'] / df['updated_weights'].sum()
)

print(df)

# -----
# Step 10: Create Cumulative Sum
# -----

df['cumsum_upper'] = np.cumsum(df['normalized_weights'])

df['cumsum_lower'] = (
    df['cumsum_upper'] - df['normalized_weights']
)

print(df[['X1', 'X2', 'label', 'weights',
            'y_pred', 'updated_weights',
            'normalized_weights',
            'cumsum_lower', 'cumsum_upper']])

# -----
# Step 11: Create New Dataset
# -----

def create_new_dataset(df):
    indices = []

    for i in range(df.shape[0]):
        a = np.random.random()

        for index, row in df.iterrows():
            if row['cumsum_upper'] > a and a > row['cumsum_lower']:
                indices.append(index)

    return indices

index_values = create_new_dataset(df)

print("Selected Indices :", index_values)

# -----
# Step 12: Second Dataset
# -----

second_df = df.iloc[index_values, [0, 1, 2, 3]]

print(second_df)

# -----
# Step 13: Train Second Weak Learner
# -----

dt2 = DecisionTreeClassifier(max_depth=1)

X2 = second_df.iloc[:, 0:2].values
y2 = second_df.iloc[:, 2].values

dt2.fit(X2, y2)

# Prediction

```

```
second_df['y_pred'] = dt2.predict(X2)

print(second_df)

# -----
# Step 14: Plot Second Tree
# -----

plt.figure(figsize=(6,4))
plot_tree(dt2, filled=True)
plt.show()

# -----
# Step 15: Plot Decision Region
# -----

plot_decision_regions(X2, y2, clf=dt2, legend=2)
plt.show()

# -----
# Step 16: Calculate Alpha2
# -----

alpha2 = calculate_model_weight(0.1)

print("Alpha2 :", alpha2)

# -----
# Step 17: Final Prediction Example
# -----

query = np.array([1,5]).reshape(1,2)

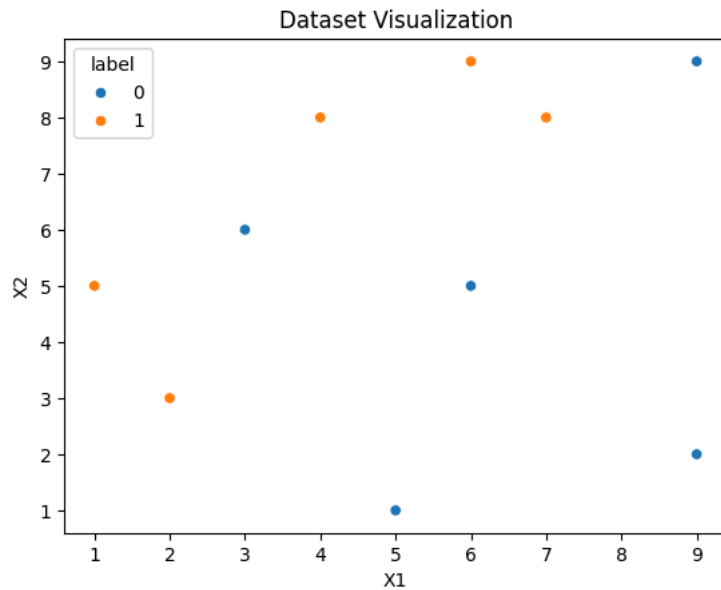
pred1 = dt1.predict(query)
pred2 = dt2.predict(query)

print("Prediction from DT1 :", pred1)
print("Prediction from DT2 :", pred2)

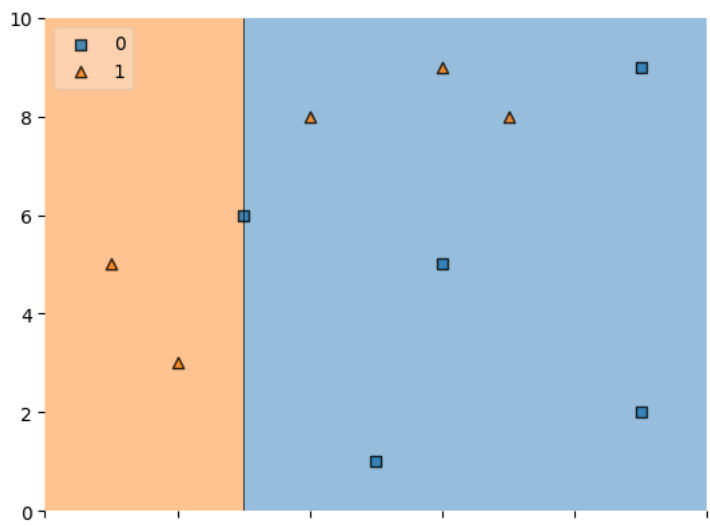
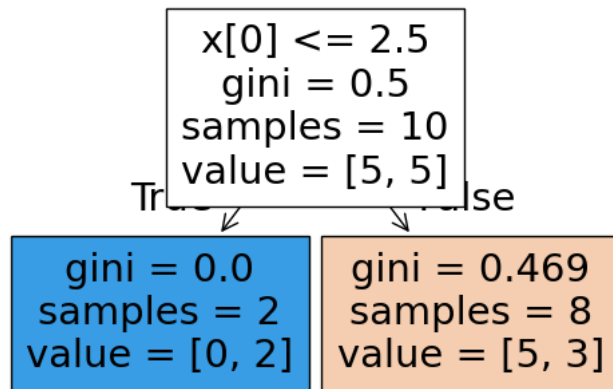
final_prediction = alpha1 * pred1 + alpha2 * pred2

print("Final Prediction :", np.sign(final_prediction))
```


	X1	X2	label
0	1	5	1
1	2	3	1
2	3	6	0
3	4	8	1
4	5	1	0
5	6	9	1
6	6	5	0
7	7	8	1
8	9	9	0
9	9	2	0



	X1	X2	label	weights	y_pred
0	1	5	1	0.1	1
1	2	3	1	0.1	1
2	3	6	0	0.1	0
3	4	8	1	0.1	0
4	5	1	0	0.1	0
5	6	9	1	0.1	0
6	6	5	0	0.1	0
7	7	8	1	0.1	0
8	9	9	0	0.1	0
9	9	2	0	0.1	0



0		2		4		6		8		10	
Alpha1 : 0.42364893019360184											
X1	X2	label	weights	y_pred	updated_weights	normalized_weights					
0	1	5	1	0.1	1	0.065508		0.071475			